Convolutional Neural Networks for NLP

From spatial gradients to semantic n-gram extraction

James Pustejovsky

March 2026

Graduate Seminar in Computer Science/Computational Linguistics

The Geometric Foundation

The Failure of the Fully Connected (MLP) Paradigm

- **High-Dimensional Inputs:** A $224 \times 224 \times 3$ image has 150,528 inputs.
- **Parameter Explosion:** A single hidden layer of 1,000 neurons $\approx$ 150M parameters.
- **Statistical Inefficiency:** MLPs treat pixel (i, j) and $(i + 1, j)$ as independent variables.

Inductive Bias: Translation Invariance
A feature (e.g., a vertical edge or the word "not") has the same identity regardless of its absolute coordinate in space or time.

Defining the Discrete 2D Convolution

The output at position (i, j) is the element-wise product sum of the input I and kernel K:

$$(I * K)_{i,j} = \sum_m \sum_n I(i+m, j+n) \cdot K(m, n)$$

- **Local Connectivity:** Neurons only "see" neighbors within a $k \times k$ window.
- **Weight Sharing:** The same weights K are applied across the entire input.

Feature Extraction: The Sobel Operator

We detect boundaries by calculating the **Intensity Gradient**:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- G_x detects horizontal transitions (vertical lines).
- G_y detects vertical transitions (horizontal lines).
- $|\nabla G| = \sqrt{G_x^2 + G_y^2}$ provides the total edge magnitude.

The Mechanics of Sizing: Padding and Stride

- **Stride (S):** The step size of the kernel. $S = 2$ reduces spatial resolution by half.
- **Padding (P):** Adding zeros to the border.
  - *Valid ($P = 0$):* Output size $W - K + 1$. Edges are lost.
  - *Same ($P = \frac{K-1}{2}$):* Output size matches Input size.

The Output Dimension Formula

$$W_{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

1D Convolutions in NLP

Mapping Space to Time: 1D Sequences

In NLP, our "Image" is a sentence matrix $X \in \mathbb{R}^{n \times d}$ (Length $\times$ Embedding Dim).

- **Temporal Convolution:** The kernel width is fixed to d (full embedding).
- **Window Size (h):** The number of words the kernel "sees" at once.
- **Result:** A feature map of length $n - h + 1$.

Kernel Size as N-gram Order

A 1D kernel acts as a soft n-gram detector:

- $h = 2$**:** Bigram detector (e.g., "not happy").
- $h = 3$**:** Trigram detector (e.g., "was not happy").

Unlike traditional n-grams, CNNs use **distributed representations** (embeddings), allowing them to recognize "not happy" and "never joyful" with the same filter.

Max-Over-Time Pooling

How do we handle variable-length sentences for a fixed-size classifier?

$$\hat{c} = \max\{c_1, c_2, \ldots, c_{n-h+1}\}$$

- **Length Invariance:** The network finds the "strongest" feature regardless of where it appears.
- **Saliency:** Discards noise; only the highest activation is passed forward.

The Kim (2014) Architecture

- **Input:** Sentence embeddings (Static vs. Non-static).
- **Multi-Filter Bank:** 100 filters for $h = 3, 4, 5$.
- **Concatenation:** Feature maps are pooled and concatenated into a single vector.
- **Output:** Softmax layer for classification (Sentiment, Intent).

Learning and Modern Frontiers

How Kernels Learn: Shared-Weight Backpropagation

- The gradient $\frac{\partial \mathcal{L}}{\partial W}$ is calculated for every position where the kernel was applied.
- **Weight Sharing:** Because the same W is used everywhere, the final update is the **sum of gradients** across all spatial/temporal steps.
- This forces the kernel to be a "Global Detector."

CNNs vs. Transformers in 2026

CNN (Local)

- Linear complexity $O(n)$.
- High efficiency on edge devices.
- Better for small datasets.

Transformer (Global)

- Quadratic complexity $O(n^2)$.
- Captures long-range dependencies.
- Requires massive pre-training.

Current SOTA trend: ConvNeXt architectures (CNNs with large 7×7 kernels) match Transformers in performance.